



Real Production Failures (Very Important)



Case 1: Netflix – Content Metadata Search Failure

✖ Problem

- Huge content metadata (movies, genres, tags)
- Used embedding-based retrieval

⚠ What went wrong:

- Query: *“dark psychological thriller with mind-bending plot”*
- Retrieved:
 - Random thrillers
 - Missing actual relevant titles

● Root Cause:

- Embeddings matched **keywords**, not **intent**
- No reasoning about:
 - “psychological”
 - “mind-bending”

★ Impact:

- Poor recommendation quality
- Lower user engagement



Case 2: Legal AI Systems (Contract Analysis)

✖ Problem

- Legal docs contain:
 - cross-references
 - clauses
 - dependencies

⚠ Example:

- Clause 4 → refers to Clause 18
- Vector RAG retrieves only Clause 4

● Root Cause:

- Chunk-based retrieval breaks relationships

★ Impact:

- Wrong legal interpretation
- High risk (compliance / lawsuits)



Case 3: Finance (Risk & Compliance Systems)

✖ Problem

- Financial reports = structured + interdependent

⚠ Example:

- Query: “Why did revenue drop in Q3?”
- Retrieved:
 - Revenue numbers
 - BUT NOT:
 - macro factors
 - cost drivers

● Root Cause:

- Similarity ≠ causation
- No reasoning

★ Impact:

- Wrong business decisions

🧠 Core Lesson

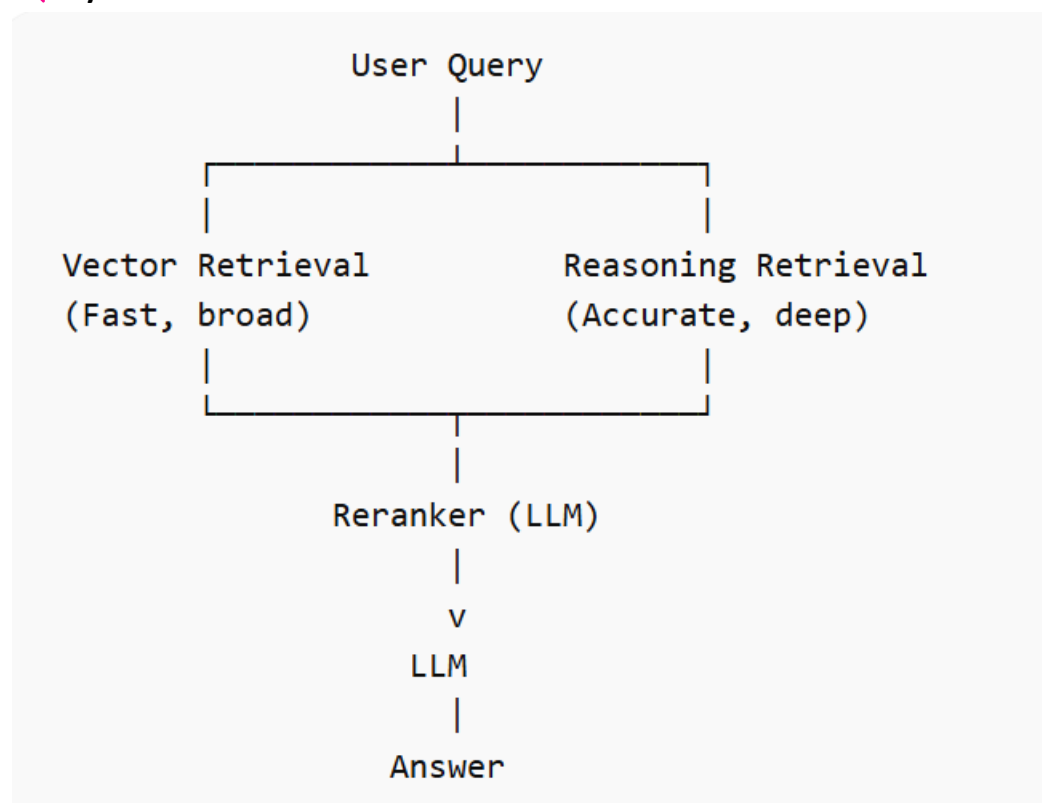
Vector RAG fails when:

- relationships matter
- reasoning is required
- context is distributed

🚀 2 Hybrid RAG (Industry Standard 2025–2026)

👉 Nobody uses pure vector OR pure vectorless in production now.

🧠 Hybrid Architecture



🔥 Components Breakdown

1 Vector Retrieval (Recall Layer)

- Fast
- Gets **candidate chunks**
- High recall, low precision

2 Reasoning Layer (Precision Layer)

- Tree traversal / Page Index
- Multi-hop reasoning
- Context stitching

3 Reranker (MOST IMPORTANT)

- LLM ranks relevance
- Filters noise

🌟 Production Stack Example

- Embeddings: Ollama / OpenAI
- Vector DB: Qdrant
- Reasoning: LLM (GPT / Claude / local)
- API: FastAPI
- Orchestration: LangChain / custom

⚡ Why Hybrid Wins

Capability	Vector	Vectorless	Hybrid
Speed	✓	✗	✓
Accuracy	✗	✓	✓ ✓
Scale	✓	✗	✓
Reasoning	✗	✓	✓



3 Code Comparison (VERY IMPORTANT)

A. Vector RAG (Classic)

Step 1: Embed documents

```
from langchain.embeddings import OllamaEmbeddings
from langchain.vectorstores import Qdrant
```

```

embeddings = OllamaEmbeddings(model="nomic-embed-text")

vector_db = Qdrant.from_texts(
    texts=documents,
    embedding=embeddings
)

# Step 2: Query
query = "Why did revenue drop?"

query_vector = embeddings.embed_query(query)

docs = vector_db.similarity_search(query, k=5)

# Step 3: LLM generation
context = "\n".join([doc.page_content for doc in docs])

prompt = f"""
Answer using context:
{context}

Question: {query}
"""

response = llm.invoke(prompt)

```

B. Vectorless RAG (Page Index Style)

```

# Step 1: Build Tree Index
def build_tree(documents, llm):
    tree = {}

    for doc in documents:
        structure = llm.invoke(f"""
Extract structure:
- sections
- topics
- summary

Document:
{doc}

```

```

        """
    )

    tree[doc.id] = structure

    return tree

# Step 2: Query (Reasoning)
def query_tree(tree, query, llm):
    relevant_nodes = llm.invoke(f"""
    Given this tree:
    {tree}

    Find relevant sections for query:
    {query}
    """)

    return relevant_nodes

# Step 3: Fetch + Answer
def generate_answer(nodes, original_docs, llm):
    context = fetch_original_content(nodes, original_docs)

    return llm.invoke(f"""
    Answer based on:
    {context}
    """)

```

C. Hybrid RAG (🔥 Real Production)

```

# Step 1: Vector Retrieval
docs = vector_db.similarity_search(query, k=10)

# Step 2: Reasoning Filter
filtered_docs = llm.invoke(f"""
Select only relevant docs:

Docs:
{docs}

```

Query:

```
{query}  
"""
```

Step 3: Optional Tree Reasoning

```
structured_context = build_dynamic_context(filtered_docs, llm)
```

Step 4: Final Answer

```
response = llm.invoke(f"""
```

Use this context:

```
{structured_context}
```

Answer:

```
{query}  
""")
```

Final Insight (Senior-Level)

 Evolution of RAG:

Naive → Vector → Hybrid → Agentic RAG

 Future direction:

- Reasoning-first retrieval
- Multi-hop queries
- Agent-based search